

Game Engines – Part 2/2



Image sources: <https://jackbrummet.blogspot.de/2007/10/video-games-why-waste-good-technology.html>

The Main Loop

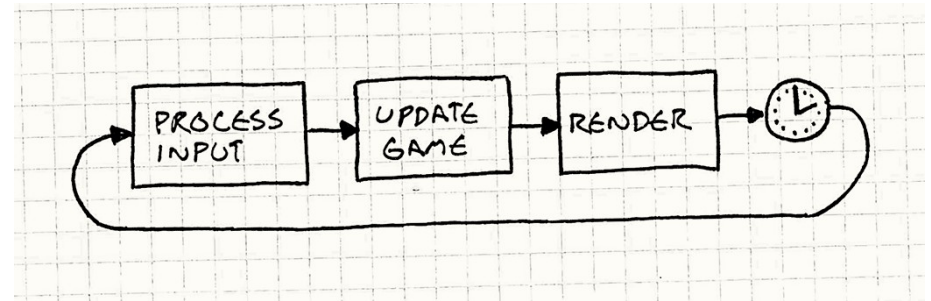


Image sources: <http://gameprogrammingpatterns.com/game-loop.html>, <https://nesterthelark.wordpress.com/2011/09/06/gallery-mode-capturing-sonic-the-hedgehog/>

Games are pretty demanding...



- Games are constantly processing, even if the player does nothing
 - Update all subsystems
 - Render the entire screen
- Subsystems need to be updated at different rates
 - We have multiple deadlines to meet
 - Rendering (~30 frames / second)
 - Physics (~120 updates / second)
 - Artificial Intelligence (~think once per second)
 - Audio Library (~60 updates / seconds)
- The simplest way to update our engine's subsystems is using a single loop, the so called game loop
 - The master loop that services every subsystem in the engine

A Simple Example: Pong



```
void main() // Pong
{
    initGame();
    while (true) // game loop
    {
        readHumanInterfaceDevices();
        if (quitButtonPressed())
            break; // exit the game loop
        movePaddles();
        moveBall();
        collideAndBounceBall();
        if (ballImpactedSide(LEFT_PLAYER))
        {
            incrementScore(RIGHT_PLAYER); resetBall();
        }
        else if (ballImpactedSide(RIGHT_PLAYER))
        {
            incrementScore(LEFT_PLAYER); resetBall();
        }
        renderPlayfield();
    }
}
```

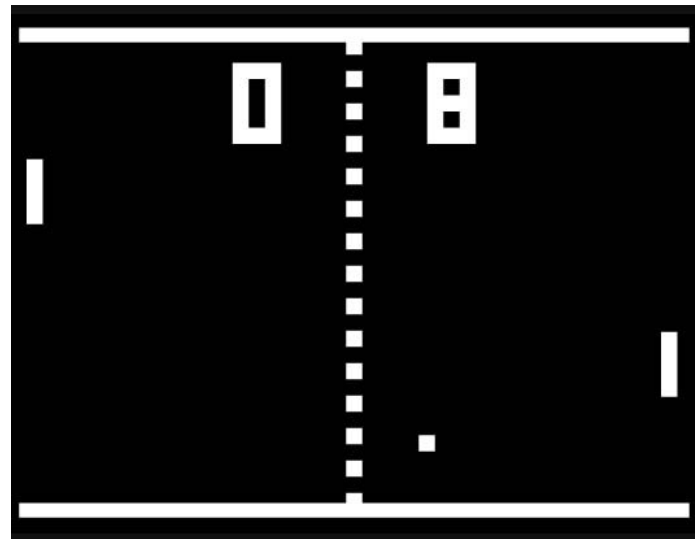


Image sources: <https://en.wikipedia.org/wiki/Pong>

Game Loop Architectural Styles (1/3)



- Windows Message Pumps
 - The basic idea is to service Windows messages whenever they arrive and to service the game engine only when no Windows messages are pending.
- “Playing nice with the OS.”

```
while (true)
{
    // Service any and all pending Windows messages.
    MSG msg;
    while (PeekMessage(&msg, NULL, 0, 0) > 0)
    {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    // No more Windows messages to process – run one
    // iteration of our "real" game loop.
    RunOneIterationOfGameLoop();
}
```

Game Loop Architectural Styles (2/3)



- Callback-Driven Frameworks
 - In a framework-based rendering engine or game engine, the main game loop has been written for us, but it is largely empty.

```
while (true)
{
    for (each frameListener)
    {
        frameListener.frameStarted();
    }
    renderCurrentScene();
    for (each frameListener)
    {
        frameListener.frameEnded();
    }
    finalizeSceneAndSwapBuffers();
}
```

```
// . . .
```

```
class MyFrameListener: public FrameListener { /*. . . */ }
```

Game Loop Architectural Styles (3/3)



- Event-Based Updating
 - An event is any interesting change in the state of the game or its environment.
 - Most game engines have an event system, which permits various engine subsystems to register interest in particular kinds of events and to respond to those events when they occur.
 - Subsystems send and receive events.
 - Events are categorized and eventually prioritized.
 - Events can usually carry user-defined data.
- A game's event system is usually very similar to the event/messaging system underlying virtually all graphical user interfaces.
- Some game engines leverage their event system in order to implement the periodic servicing of some or all of their subsystems.
 - Event system must permit events to be posted into the future. (delayed)

Measuring and Dealing with Time (1/2)



- Frame Rate and Time Deltas
 - The frame rate of a real-time game describes how rapidly the sequence of still 3D frames is presented to the viewer.
 - Typically measured in frames per second (FPS) or Hertz (Hz)
 - The amount of time that elapses between frames is known as the frame time, time delta, or delta time.
 - Represented mathematically by the symbol Δt
 - Game at 30 FPS $\rightarrow 1/30 \text{ sec} = 33.33 \text{ ms}$
 - Game at 60 FPS $\rightarrow 1/60 \text{ sec} = 16.67 \text{ ms}$
 - To really know how much time has elapsed during one iteration of the game loop, we need to measure it.

Measuring and Dealing with Time (2/2)



- From Frame Rate to Speed
 - Let's imagine that we want to make a spaceship fly through our game world at a constant speed of 40 meters per second.
 - One simple way to accomplish this is to multiply the ship's speed v (measured in meters per second) by the duration of one frame Δt (measured in seconds)
 - Change in position: $\Delta x = v \times \Delta t$
 - Position next frame: $X2 = X1 + \Delta x = X1 + v \times \Delta t$
 - This is actually a simple form of numerical integration known as the explicit Euler method.
 - Works well as long as the speeds of our objects are roughly constant.
 - To handle variable speeds, we need to resort to somewhat more complex integration methods.
 - No matter what integration method we use, it is safe to say that the perceived speeds of the objects in a game are dependent upon the frame duration, Δt .

Old-School CPU-Dependent Games



- No attempt was made to measure how much real time had elapsed during the game loop.
 - Speeds of objects were specified directly in units per frame
- Perceived speeds of the objects in these games were entirely dependent upon the frame rate
 - Faster CPU → faster running game (CPU-dependent games)
 - Older PCs provided a “Turbo” button to support these kinds of games
- This approach is still used in games today
 - As long as the frame-rate is constant, all speeds are correct
 - Consoles have homogeneous hardware
 - Frame rate can be synchronised with the TV

Updating Based on Elapsed Time (1/2)



- To make our games CPU-independent, we must measure Δt
 - Measure time at the beginning of the frame: t_1
 - Measure time at the end of the frame: t_2
 - $\Delta t = t_2 - t_1$
- There is one big problem with this technique:
 - We use Δt taken during frame k as an estimate of the duration of the upcoming frame $(k+1)$!
 - “Past performance is not a guarantee of future results.”
 - Frame-rate spike: frame k is much shorter or longer than frame $(k+1)$
 - Example:
 - Our physics system needs to be updated every 33.3 ms
 - The last frame k took 57ms $\rightarrow$ We assume we have to update twice in $(k+1)$
 - The next frame will be at least as bad, and possibly worse!
 - This can cause a “viscious cycle” of poor frame times

Updating Based on Elapsed Time (2/2)



- Using a Running Average
 - Game loops tend to have at least some frame-to-frame coherency
 - One reasonable approach is to average the frame-time measurements over a small number of frames
 - The game can adapt to a varying frame rate
 - The effects of momentary performance spikes are softened
 - $\Delta t_{k+1} = (\Delta t_k + \Delta t_{k-1} + \Delta t_{k-2} + \dots) / n$
- Governing the Frame Rate
 - We try to guarantee that every frame's duration will be exactly the same
 - Measure time as before
 - Frame too short → sleep until target frame time is reached
 - Frame too long → wait for one more entire frame to elapse
 - Advantages: Looks better, reliability, some subsystems operate best when updated at a constant rate
 - Drawbacks: Frame rate will halve due to frequent “slow” frames

Multiprocessor Game Loops (1/5)



- **One Thread per Subsystem**
 - Engine subsystems to run in separate threads
 - Master thread would control and synchronize
 - Also most of the high-level logic
 - This is a simple example of *task parallelism*
- **Shortcomings**
 - Number of subsystems != number of cores
 - Each engine subsystem requires a different amount of processing each frame
 - Potentially lots of idle time!
 - some engine subsystems depend on the data produced by others
 - E.g. the rendering and audio subsystems depend on collision and physics

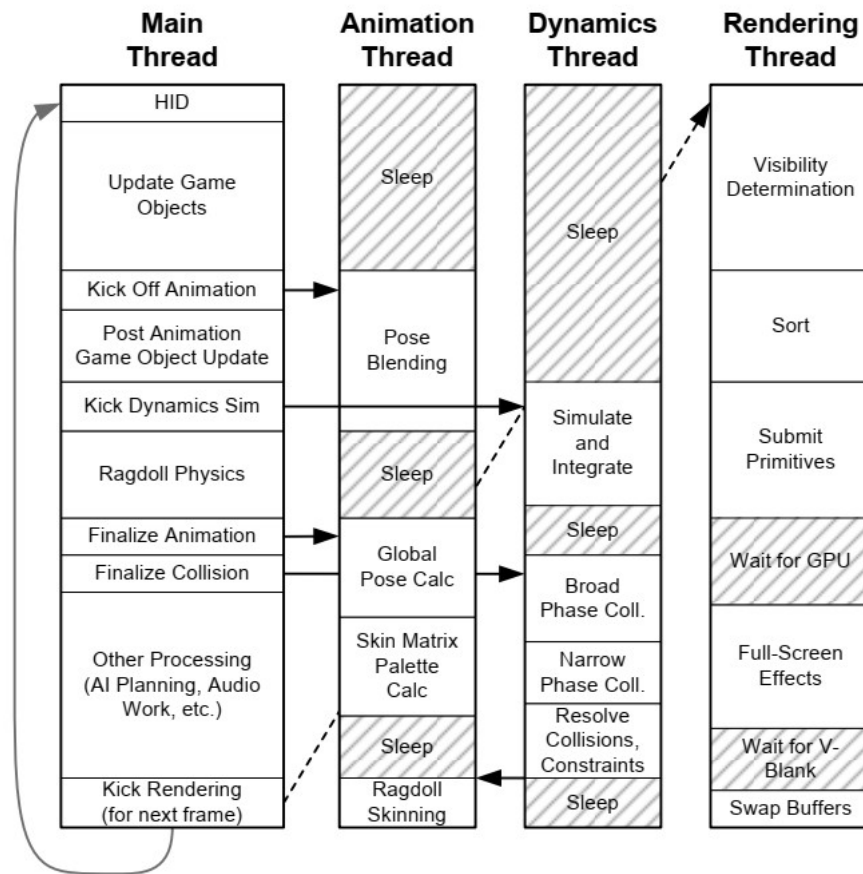


Image sources: Game Engine Architecture, Taylor & Francis Ltd., 2018

Multiprocessor Game Loops (2/5)



- **Scatter/Gather (aka Fork/Join)**
 - Many of the tasks the game loop are data-intensive
 - Perform such tasks using a *divide-and-conquer* approach
 - E.g. subdivide 9000 ray casts into 6 x 1500
 - This approach is a form of *data parallelism*
 - *Scatter*: Subdivide the data into smaller sub-units
 - *Gather*: Combine or finalize the data
- **Considerations**
 - Might happen multiple times within the loop
 - Master thread spawns/joins worker threads
 - Threads could also be pre-spawned
 - Can be combined with SIMD
 - Threads might have different run times
 - Not all cores are busy all the time within the loop

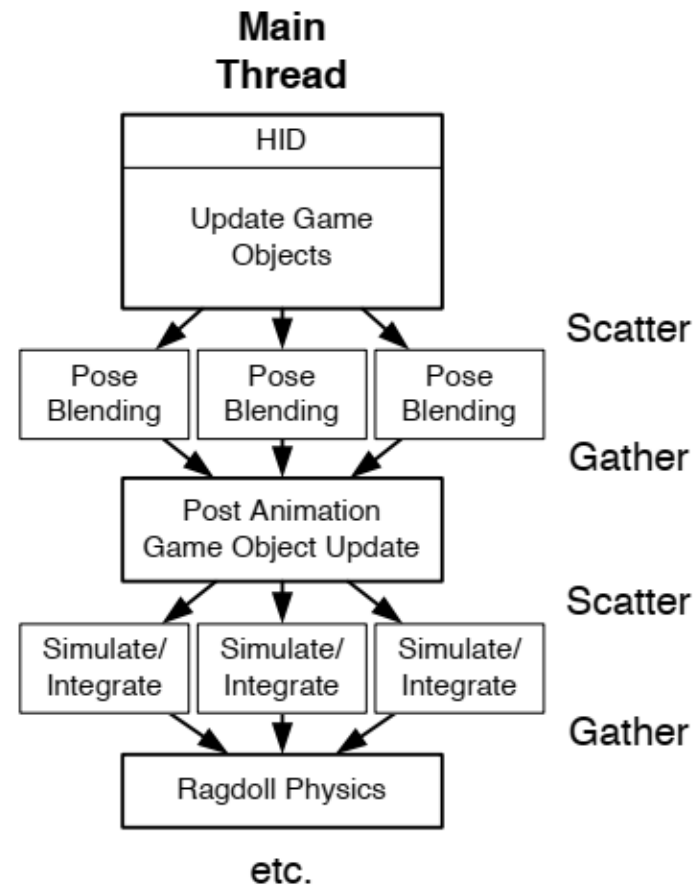


Image sources: Game Engine Architecture, Taylor & Francis Ltd., 2018

Multiprocessor Game Loops (3/5)



- **Job Systems**

- ...are a general-purpose systems for executing arbitrary units of work typically called *jobs*, across multiple cores.
 - Subdivide each iteration of the game loop into a relatively large number of independent jobs
 - Submit them to the job system for execution
 - It schedules those jobs across the available cores
 - Jobs are executed by worker threads in a thread pool
 - This architecture scales up or down naturally to hardware with any number of CPU cores

- **Considerations**

- Requires a solid interface (API) for the jobs
- Synchronization can be tricky (and potentially slow)
- Splitting up the work may be challenging

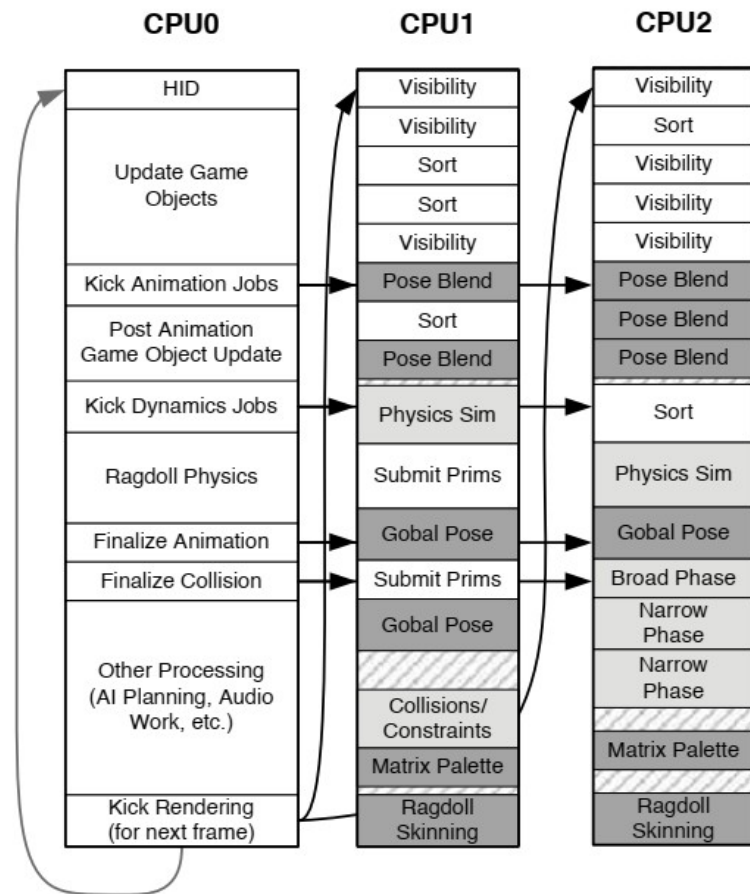


Image sources: Game Engine Architecture, Taylor & Francis Ltd., 2018

Multiprocessor Game Loops (4/5)



- Job Systems: Advanced considerations
 - Some jobs may depend on other ones and require “completion”
 - Jobs could be implemented as *coroutines*, *fibers*, etc.
 - Can then be put to sleep and re-awaken later
 - Jobs may need to be grouped and/or prioritized
 - Different synchronization primitives need to be available
 - E.g. locks, semaphores, barriers, etc.
 - Requirement for Visualization and Profiling Tools
- “Parallelizing the Naughty Dog Engine” (GDC 2015)
 - <https://bit.ly/2H6v0J4>
 - <https://bit.ly/2ETr5x9>



Image sources: Game Engine Architecture, Taylor & Francis Ltd., 2018

Multiprocessor Game Loops (5/5)



<https://www.youtube.com/watch?v=QIKXPBFh5BM>